

گزارش پروژه کامپایلر - فاز دوم

در این پروژه یک تحلیلگر نحوی (Parser) طراحی و پیاده‌سازی شده است که با استفاده از CUP و در برخی قسمت‌ها JFlex، توانایی تشخیص و پردازش ساختارهای مختلف زبان برنامه‌نویسی جاوا را دارد. این فاز در ادامه‌ی فاز اول پروژه اصول طراحی کامپایلر انجام شده و مطابق با نیازمندی‌های تعریف شده و خواسته شده در پروژه پیاده‌سازی شده است.

در ادامه گزارشی از قسمت‌های مهم کد ارائه شده که در فایل Java12.cup موجود هستند. همچنین فایل in.Java مربوط به ورودی است.

در انتهای گزارش نیز تصویر ورودی و خروجی مرتبط با آن موجود است.

((بخش‌های اصلی کد))

1. جدول نمادها (Symbol Table)

در ابتدای کد دو ساختار داده تعریف شده است:

- `Map<String, String> symbolTable` برای نگهداری نوع متغیرها.
 - `Map<String, Number> varValues` برای ذخیره مقدار عددی متغیرها.
- این دو جدول نقش مدیریت متغیرها، نوع‌ها و مقادیر اولیه آن‌ها را دارند.

2. توابع عملیات ریاضی

متدهای `addNumbers`, `subNumbers`, `mulNumbers`, `divNumbers`, `modNumbers` عملیات اصلی روی داده‌های عددی را پشتیبانی می‌کنند. این توابع به گونه‌ای طراحی شده‌اند که انواع داده‌های مختلف (`int`, `float`, `double`, `long`) را مدیریت کنند. برای مثال در `addNumbers` اگر یکی از عملوندها از نوع `Double` باشد نتیجه نیز `Double` محاسبه می‌شود. مثالی از این قسمت در پروژه در فایل `Java12.cup`:

```
172 private Number divNumbers(Number_a, Number_b) {
173     if (a.doubleValue() == 0.0) {
174         throw new ArithmeticException("Division by zero");
175     }
176     if (a instanceof Double || b instanceof Double) {
177         return Double.valueOf(a.doubleValue() / b.doubleValue());
178     } else if (a instanceof Float || b instanceof Float) {
179         return Float.valueOf(a.floatValue() / b.floatValue());
180     } else if (a instanceof Long || b instanceof Long) {
181         return Long.valueOf(a.longValue() / b.longValue());
182     } else {
183         return Integer.valueOf(a.intValue() / b.intValue());
184     }
185 }
186
187 private Number modNumbers(Number_a, Number_b) {
188     if (a instanceof Double || b instanceof Double) {
189         return Double.valueOf(a.doubleValue() % b.doubleValue());
190     } else if (a instanceof Float || b instanceof Float) {
191         return Float.valueOf(a.floatValue() % b.floatValue());
192     } else if (a instanceof Long || b instanceof Long) {
193         return Long.valueOf(a.longValue() % b.longValue());
194     } else {
195         return Integer.valueOf(a.intValue() % b.intValue());
196     }
197 }
```

یک درخت نحوی انتزاعی (AST) نیز ساخته می‌شود. برای هر عملگر دودویی مانند $+$, $-$, $*$, $/$ گره‌ای از نوع `BinaryOpNode` ایجاد می‌گردد که عملوندهای چپ و راست را در قالب `VariableNode` یا `NumberNode` نگهداری می‌کند. این ساختار امکان نمایش گرافیکی و حفظ تقدم عملگرها (مثلاً ضرب و تقسیم قبل از جمع) را فراهم می‌کند. خروجی نمونه‌ی آن در زمان اجرا به صورت زیر چاپ می‌شود:

AST: (x + (y * 3))

```

1388 ArithmeticResult left = (ArithmeticResult) a;
1389 ArithmeticResult right = (ArithmeticResult) m;
1390 Number resultValue = addNumbers(left.value, right.value);
1391 String resultOp =
1392     "plus Foundd"+"(" + left.operation + " + " + right.operation + ")"+ "=" + resultValue;
1393 try {
1394     ASTNode leftNode = (left.isVariable()) ?
1395         new VariableNode(left.getVarName()) :
1396         new NumberNode(left.value);
1397     ASTNode rightNode = (right.isVariable()) ?
1398         new VariableNode(right.getVarName()) :
1399         new NumberNode(right.value);
1400
1401     ASTNode ast = new BinaryOpNode("+", leftNode, rightNode);
1402     System.out.println("AST: " + ast.toString());

```

3. کلاس‌های کمکی برای مدیریت ساختارها

- `VariableInfo`: نگهداری نام و مقدار اولیه متغیر.
- `FunctionInfo`: ذخیره اطلاعات توابع شامل نوع بازگشتی، نام و لیست پارامترها.

```

215 class VariableInfo {
216     String name;
217     Object initialValue;
218
219     public VariableInfo(String name, Object initialValue) {
220         this.name = name;
221         this.initialValue = initialValue;
222     }
223
224     public String toString() {
225         return "Variable: " + name + ", Initial Value: " + (initialValue != null ? initialValue.toString() : "null");
226     }
227 }
228 class FunctionInfo {
229     String returnType;
230     String name;
231     java.util.List<ParameterInfo> parameters;
232
233     public FunctionInfo(String returnType, String name, java.util.List<ParameterInfo> parameters) {
234         this.returnType = returnType;
235         this.name = name;
236         this.parameters = parameters;
237     }
238 }

```

- ParameterInfo: نگهداری اطلاعات مربوط به هر پارامتر تابع.
- ArithmeticResult: برای نمایش نتیجه و نحوه محاسبه یک عبارت ریاضی.

4. تعریف ترمینال‌ها و نان‌ترمینال‌ها

بخش عمده کد به تعریف ترمینال‌ها (کلمات کلیدی و عملگرها) و نان‌ترمینال‌ها (قوانین نحوی) اختصاص یافته است. برای مثال:

- پایانه‌ها (ترمینال): IF, ELSE, FOR, WHILE, CLASS, INT_LITERAL و ...
- غیرپایانه‌ها (نان‌ترمینال): expression, statement, class_declaration, method_declaration و ...

```
398 non terminal type, primitive_type, numeric_type;
399 non terminal integral_type, floating_point_type;
400 non terminal reference_type;
401 non terminal class_or_interface_type;
402 non terminal class_type, interface_type;
403 non terminal array_type;
```

```
326 terminal method_invocation_statement;
327 terminal assignment_statement;
328 terminal Float Float_LITERAL;
329 terminal Double Double_LITERAL;
330 terminal Integer INT_LITERAL;
331 terminal BOOLEAN; // primitive_type
332 terminal BYTE, SHORT, INT, LONG, CHAR; // integral_type
333 terminal FLOAT, DOUBLE; // floating_point_type
334 terminal LBRACK, RBRACK; // array_type
335 terminal DOT; // qualified name
```

5. پشتیبانی از ساختارهای زبان

- **تعریف متغیر:** هنگام رسیدن به دستور تعریف متغیر، نوع و نام آن چاپ می‌شود. اگر مقدار اولیه داشته باشد، مقدار نیز نمایش داده می‌شود.
- **ساختارهای شرطی:** پشتیبانی از if, else, else if همراه با چاپ نوع ساختار تشخیص داده شده. (تصویر پایین)
- **حلقه‌ها:** شامل for, while, do while. در هر مورد پیام شناسایی ساختار چاپ می‌شود.
- **توابع:** نام، نوع بازگشتی و لیست پارامترها چاپ می‌شود.
- **کلاس‌ها:** نام کلاس تشخیص داده و نمایش داده می‌شود.
- **عملیات ریاضی:** محاسبات انجام و نتیجه نهایی همراه با جزئیات نمایش داده می‌شود.

- فراخوانی تابع: نام تابع و آرگومان‌های ارسال شده چاپ می‌شود.

```

1108 for_statement ::=
1109     FOR LPAREN for_init_opt:in SEMICOLON expression_opt:cond SEMICOLON
1110     | for_update_opt:update RPAREN statement:stmt
1111     {:
1112         System.out.println("For statement");
1113         RESULT = new ToString("for");
1114     :}
1115 ;
1116 for_statement_no_short_if ::=
1117     FOR LPAREN for_init_opt SEMICOLON expression_opt SEMICOLON
1118     | for_update_opt RPAREN statement_no_short_if
1119     ;
1120 for_init_opt ::=
1121     | for_init
1122     ;
1123 for_init ::= statement_expression_list
1124     | local_variable_declaration
1125     ;
1126 for_update_opt ::=
1127     | for_update
1128     ;
1129 for_update ::= statement_expression_list

```

6. مدیریت خطا

دو متد `report_fatal_error` و `report_error` برای گزارش خطاهای نحوی در نظر گرفته شده‌اند که پیام مناسب به همراه موقعیت خطا را چاپ می‌کنند.

```

211 public void report_fatal_error(String message, Object info) {
212     report_error(message, info);
213     throw new RuntimeException("Fatal Syntax Error");
214 }

```

نکات مهم و خروجی‌ها

مطابق نیازمندی پروژه:

- برای تعریف متغیر، نوع، نام و مقدار چاپ می‌شود.
- برای شرط یا حلقه، نوع آن شناسایی و نمایش داده می‌شود.
- برای تابع، نوع خروجی و پارامترها گزارش می‌شوند.
- برای کلاس، نام کلاس چاپ می‌شود.
- برای عملیات ریاضی، مقدار نهایی محاسبه و نمایش داده می‌شود.
- برای فراخوانی تابع، نام تابع و لیست آرگومان‌ها چاپ می‌شود.

تصویر نمونه ورودی/خروجی

در گزارش اصلی باید تصویری از یک ورودی نمونه (کدی شامل تعریف کلاس، متغیر، حلقه، شرط و تابع) و خروجی تولیدشده توسط تحلیلگر درج شود. نمونه ورودی در فایل in.Java و خروجی که کامپایلر داده در تصویر زیر است:

```
class Test {  
    float a = 256.4;  
    public void main(int x) {  
        int x;  
        x=2+3;  
        for (int i = 0; i < 2; i++) {int a;}  
        while (i>83){}  
        if(i<=4){}  
    }public void test() {  
        int z = 5;  
        main(z);  
        int y = z *2;  
    }  
}
```

JavaParser ×

Parsing [in.java]

Variable: a, Type: float, Initial Value: 256.4

function Variable: x, Initial Value: null

Assignment: x = Expression: plus Founnd(2 + 3)=5

function Variable: a, Initial Value: null

for statement

While loop found

if statement

FunctionDef:Function: main, Return type: void, Parameters: int x

function Variable: z, Initial Value: 5

funcall main send param int z

function Variable: y, Initial Value: Expression: multiplication Founnd(z * 2)=10

FunctionDef:Function: test, Return type: void, Parameters: none

ClassDef:Test

No errors.

Process finished with exit code 0